

Bit, Byte ve Integer

BIL-304: Bilgisayar Mimarisi

Dersi veren öğretim üyesi:

Dr. Öğr. Üyesi Fatih Gökçe

Ders kitabına ait sunum dosyalarından adapte edilmiştir: <http://csapp.cs.cmu.edu/>

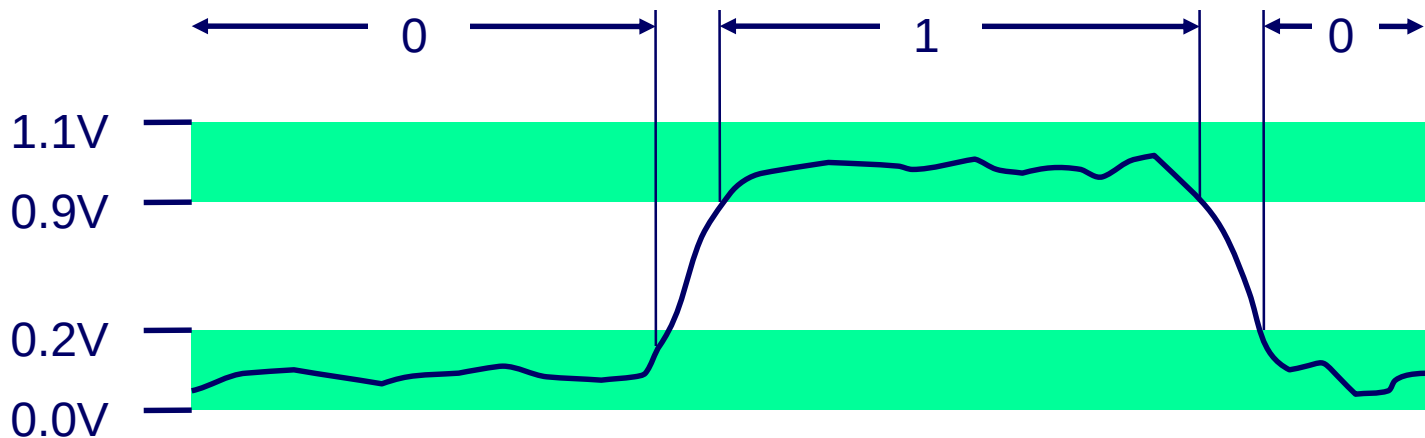
Adapted from slides of the textbook: <http://csapp.cs.cmu.edu/>

Bit, Byte ve Integer

- **Bilginin bitler ile ifade edilmesi**
- Bit düzeyinde manipülasyonlar
- **Integer**
 - unsigned ve signed
 - Dönüştürme (conversion), tip dönüşümü (casting)
 - Genişletme (expanding), kırpma (truncating)
 - Toplama, işaret değiştirme (negation), çarpma, kaydırma
- Bilginin hafızada saklanma şekilleri, pointer'lar, karakter dizileri (string'ler)

Herşey bit ile ifade edilir

- Her bit 0 veya 1'dir.
- Bit gruplarının değişik şekillerde kodlanması/anlamlandırılması ile
 - Bilgisayarlar ne yapacaklarına karar verirler (komutlar, opcode lar..)
 - ... ve sayıları, karakter dizilerini, vs ifade edip manipüle ederler.
- **Neden bit? Elektronik sistem olarak uygulama kolaylığı**
 - İki durumlu elektronik elemanlarla kaydetmek kolaydır.
 - Gürültü altında bile analog sinyallere göre daha güvenli iletilebilirler.



Byte Değerlerinin Kodlanması

■ Byte = 8 bit

- Binary 00000000₂ den 11111111₂ ye kadar.
- Onluk tabanda: 0₁₀ - 255₁₀
- Onaltılık tabanda: 00₁₆ - FF₁₆
 - 16 tabanında sayıların ifade edilmesi
 - '0' dan '9' a rakamlar ve 'A' dan 'F' e karakterler
 - C dilinde FA1D37B₁₆ şu şekillerde yazılır:
 - 0xFA1D37B
 - 0xfa1d37b

Onaltılık	Onluk	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

C'de veri tiplerinin tipik boyutları

C Veri Tipi	32-bit program	64-bit program
<code>char</code>	1	1
<code>short</code>	2	2
<code>int</code>	4	4
<code>long</code>	4	8
<code>int32_t</code>	4	4
<code>Int64_t</code>	8	8
<code>float</code>	4	4
<code>double</code>	8	8
<code>char *</code>	4	8

Bit, Byte ve Integer

- Bilginin bitler ile ifade edilmesi
- **Bit düzeyinde manipülasyonlar**
- **Integer**
 - unsigned ve signed
 - Dönüştürme (conversion), tip dönüşümü (casting)
 - Genişletme (expanding), kırpma (truncating)
 - Toplama, işaret değiştirme (negation), çarpma, kaydırma
- Bilginin hafızada saklanma şekilleri, **pointer'lar**, karakter dizileri (string'ler)

Boole Cebri

■ George Boole tarafından 19. yüzyılda geliştirilmiştir.

- Lojiğin cebirsel ifadesidir
 - “Doğru” 1 ile “Yanlış” 0 ile kodlanır.

Ve (And)

- Hem $A=1$ hem de $B=1$ ise $A \& B = 1$

$\&$	0	1
0	0	0
1	0	1

Veya (Or)

- $A=1$ veya $B=1$ ise $A \mid B = 1$

$\mid$	0	1
0	0	1
1	1	1

Değil (Not)

- $A=0$ ise $\sim A = 1$

$\sim$	
0	1
1	0

Özel-veya (Exclusive-Or, Xor)

- $A=1$ veya $B=1$ ise fakat ikisi birden 1 değilse $A \wedge B = 1$

$\wedge$	0	1
0	0	1
1	1	0

Genel Boole Cebri

■ Bit vektörleri (bit dizileri) üzerinde işlemler

- İşlemler bit düzeyinde karşılıklı bitler arasında uygulanır

01101001	01101001	01101001	01101001
<u>&01010101</u>	<u> 01010101</u>	<u>^01010101</u>	<u>~01010101</u>
01000001	01111101	00111100	10101010

■ Boole cebrinin tüm özellikleri geçerlidir

Küme şeklinde gösterim ve Manipülasyonlar

■ Gösterim

- w genişliğinde bir A bit dizisi $\{0, \dots, w-1\}$ kümesinin bir alt kümesidir
- Eğer $j \in A$ ise $a_j = 1$ 'dir. a_j , A 'daki her bir biti göstermektedir.

▪ 01101001 $\{0, 3, 5, 6\} \rightarrow A$

▪ 76543210

▪ 01010101 $\{0, 2, 4, 6\} \rightarrow B$

▪ 76543210

■ A ve B kümeleriyle işlemler

- | | | |
|----------------------------|----------|------------------------|
| ▪ & Kesişim | 01000001 | $\{0, 6\}$ |
| ▪ Birleşim | 01111101 | $\{0, 2, 3, 4, 5, 6\}$ |
| ▪ ^ Simetrik Fark (XOR) | 00111100 | $\{2, 3, 4, 5\}$ |
| ▪ ~ Tümleyen | 10101010 | $\{1, 3, 5, 7\}$ |

C'de Bit-Düzeyinde İşlemler

- **C'de &, |, ~, ^ işlemleri bulunmaktadır.**
 - Tam sayıların sonlu bir alt kümesini gösteren tüm veri tiplerine (“integral” veri tipi) uygulanabilirler.
 - long, int, short, char, unsigned
 - Argümanları bir vektörü olarak görürler
 - Argümanlar bit-düzeyinde işleme tabi tutulurlar
- **Örnekler (Char veri tipi için)**
 - $\sim 0x41 \rightarrow 0xBE$
 - $\sim 01000001_2 \rightarrow 10111110_2$
 - $\sim 0x00 \rightarrow 0xFF$
 - $\sim 00000000_2 \rightarrow 11111111_2$
 - $0x69 \& 0x55 \rightarrow 0x41$
 - $01101001_2 \& 01010101_2 \rightarrow 01000001_2$
 - $0x69 | 0x55 \rightarrow 0x7D$
 - $01101001_2 | 01010101_2 \rightarrow 01111101_2$

C'de Lojik (Mantıksal) İşlemler

- Bit-Düzeyinde işlemlerden farklıdırlar
 - Semboller → VE: &&, VEYA: ||, DEĞİL: !
 - 0 → “Yanlış”
 - Sıfırdan farklı değer → “Doğru”
 - Ya 0 (Yanlış) ya da 1 (Doğru) değeri dönerler
 - Lojik ifadenin değeri birinci argümana bakılarak belirlenebiliyorsa ikinci argüman değerlendirilmez.
- Örnekler (char veri tipi. İşlemlerin bit-düzeyindeki karşılıklarından farklı sonuçlar çıkabildiğine dikkat ediniz!)
 - !0x41 → 0x00
 - !0x00 → 0x01
 - !!0x41 → 0x01
 - 0x69 && 0x55 → 0x01
 - 0x69 || 0x55 → 0x01 (0x69 Doğru olduğu için VEYA operatöründen sonra gelen ikinci argümanı değerlendirmeye gerek kalmaz)
 - p && *p (Null pointer erişimi hatasını önler, p=0 ise p bir Null pointer'dir, operatör VE olduğu için *p yi değerlendirmeden sonucun 0 olduğu sonucuna varılır.)
 - a && 5/a (a=0 ise ikinci argüman işletilmeden 0 döner; hiçbir zaman sıfıra bölme hatası ile karşılaşılmaz.)

C'de Lojik (Mantıksal) İşlemler

■ Bit-Düzeyinde işlemlerden farklıdırlar

- Semboller → VE: &&, VEYA: ||, DEĞİL: !

- 0 → “Yanlış”
- Sıfırdan farklı değer → “Doğru”
- Ya 0 (Yanlış) ya 1 (Doğru)
- Lojik ifadenin dikkatli değerlendirilmesi

■ Örnekler (char verileri için sonuçlar çıkabildiği)

- !0x41 → 0x00
- !0x00 → 0x01
- !!0x41 → 0x01
- 0x69 && 0x55 → 0x01
- 0x69 || 0x55 → 0x01 (0x69 Doğru olduğu için VEYA operatöründen sonra gelen ikinci argümanı değerlendirmeye gerek kalmaz)
- p && *p (Null pointer erişimi hatasını önler, p=0 ise p bir Null pointer'dir, operatör VE olduğu için *p yi değerlendirmeden sonucun 0 olduğu sonucuna varılır.)
- a && 5/a (a=0 ise ikinci argüman işletilmeden 0 döner; hiçbir zaman sıfıra bölme hatası ile karşılaşılmaz.)

DİKKAT!

&& Mantıksal, & Bit-düzeyinde VE
|| Mantıksal, | Bit-düzeyinde VEYA

Kaydırma (Shift) İşlemleri

- **Sola Kaydırma:** $x \ll y$
 - x bit vektörünü y kadar sola kaydırır.
 - Soldaki ekstra bitler atılır.
 - Sağdaki kayan bitlerin yeri 0 ile doldurulur.
- **Sağa kaydırma:** $x \gg y$
 - x bit vektörünü y kadar sağa kaydırır.
 - Sağdaki ekstra bitler atılır.
 - İki tipi vardır: Lojik ve Aritmetik
 - Lojik kaydırma
 - Soldaki kayan bitlerin yeri 0 ile doldurulur.
 - Aritmetik kaydırma
 - Soldaki kayan bitlerin yeri işaret biti ile doldurulur.
- **Tanımlanmamış durumlar**
 - Kaydırma miktarı 0'dan küçük ise veya word size'a eşit ya da word size'dan büyük ise.

Argüman x	01100010
<< 3	00010000
Lojik >> 2	00011000
Arit. >> 2	00011000

Argüman x	10100010
<< 3	00010000
Lojik >> 2	00101000
Arit. >> 2	11101000

İki tip sağa kaydırma olmasına rağmen C'de tek bir sağa kaydırma operatörü vardır, $\gg$. Birçok C compiler'ı sağa kaydırmanın tipini integer'ın türüne göre belirler; sıklıkla signed integer'lar aritmetik kaydırmaya göre, unsigned integer'lar da lojik kaydırmaya göre sağa kaydırılır.

Swap (Değiş-Tokuş)

```
void swap(int *x, int *y)
{
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
```

- Problem: Yukarıdaki iki integer'ın değerlerini değiş-tokuş etmemizi sağlayan swap fonksiyonunu geçici bir değişken kullanmadan yapabilir miyiz?
- Yol gösterme: Bit-düzeyinde XOR (^) kullanınız.
- $A \oplus A = 0$ ve $A \oplus 0 = A$ 'dır.

Xor ile Değiş-Tokuş

- Bit-düzeyinde Xor bir tür toplamadır.
- Xor'un $A \wedge A = 0$ olmasını sağlayan ekstra özelliği (Her değer kendisinin Xor ile toplamaya göre tersidir.) ve $A \wedge 0 = A$ sayesinde **funny** adlı fonksiyonu yazabiliriz.

```
void funny(int *x, int *y)
{
    *x = *x ^ *y;      /* #1 */
    *y = *x ^ *y;      /* #2 */
    *x = *x ^ *y;      /* #3 */
}
```

	*x	*y
Başlangıç	A	B
1	$A \wedge B$	B
2	$A \wedge B$	$(A \wedge B) \wedge B = A$
3	$(A \wedge B) \wedge A = B$	A
Bitiş	B	A

Bit, Byte ve Integer

- Bilginin bitler ile ifade edilmesi
- Bit düzeyinde manipülasyonlar
- **Integer**
 - **unsigned ve signed**
 - Dönüştürme (conversion), tip dönüşümü (casting)
 - Genişletme (expanding), kırpma (truncating)
 - Toplama, işaret değiştirme (negation), çarpma, kaydırma
- Bilginin hafızada saklanma şekilleri, pointer'lar, karakter dizileri (string'ler)

Integer'ların kodlanması

Unsigned (işaretsiz)

$$B2U(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$$

2 tümleyene göre signed (işaretli)

$$B2T(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

```
short int x = 15213;
short int y = -15213;
```

**İşaret
Biti**

- C'de short 2 byte uzunluğundadır.

	Decimal	Hex	Binary
x	15213	3B 6D	00111011 01101101
y	-15213	C4 93	11000100 10010011

■ İşaret Biti

- 2 tümleyene göre işaretli sayıları gösterdiğimizde en anlamlı bit işaret bitidir
 - Negatif olmayan sayılar için işaret biti 0
 - Negatif sayılar için 1'dir.

2 Tümleyen kodlama örnekleri

$x =$ 15213: 00111011 01101101
 $y =$ -15213: 11000100 10010011

Ağırlık	15213		-15213	
1	1	1	1	1
2	0	0	1	2
4	1	4	0	0
8	1	8	0	0
16	0	0	1	16
32	1	32	0	0
64	1	64	0	0
128	0	0	1	128
256	1	256	0	0
512	1	512	0	0
1024	0	0	1	1024
2048	1	2048	0	0
4096	1	4096	0	0
8192	1	8192	0	0
16384	0	0	1	16384
-32768	0	0	1	-32768
Toplam	15213		-15213	

İkili sayıların bir özelliğinin tümevarım ile ispatı

Önerme

$$1 + 1 + 2 + 4 + 8 + \dots + 2^{w-1} = 2^w$$

$$1 + \sum_{i=0}^{w-1} 2^i = 2^w$$

■ **w = 0:**

■ $1 = 2^0$

■ **w=k için doğru olduğunu kabul edelim:**

■ **w=k+1 için:**

$$\begin{aligned} & \text{■ } 1 + 1 + 2 + 4 + 8 + \dots + 2^{k-1} + 2^k = 2^k + 2^k = 2^{k+1} \\ & \quad \underbrace{\hspace{10em}}_{= 2^k} \end{aligned}$$

Nümerik Sınır (Limit) Değerler ve Bazı Özel Değerler

■ Unsigned değerler

$$\begin{array}{lcl} \blacksquare & UMin & = 0 \\ & 000\dots0 \end{array}$$

$$\begin{array}{lcl} \blacksquare & UMax & = 2^w - 1 \\ & 111\dots1 \end{array}$$

■ 2 Tümleyen Değerler

$$\begin{array}{lcl} \blacksquare & TMin & = -2^{w-1} \\ & 100\dots0 \end{array}$$

$$\begin{array}{lcl} \blacksquare & TMax & = 2^{w-1} - 1 \\ & 011\dots1 \end{array}$$

■ Özel Değerler

$$\begin{array}{lcl} \blacksquare & \text{Negatif 1} & \\ & 111\dots1 \end{array}$$

$$\begin{array}{lcl} \blacksquare & \text{Sıfır} & \\ & 000\dots0 \end{array}$$

w = 16 bit genişliği için değerler

	Decimal	Hex	Binary
UMax	65535	FF FF	11111111 11111111
TMax	32767	7F FF	01111111 11111111
TMin	-32768	80 00	10000000 00000000
-1	-1	FF FF	11111111 11111111
0	0	00 00	00000000 00000000

Farklı Word Size'lar için Değerler

	W (Word size)			
	8	16	32	64
UMax	255	65,535	4,294,967,295	18,446,744,073,709,551,615
TMax	127	32,767	2,147,483,647	9,223,372,036,854,775,807
TMin	-128	-32,768	-2,147,483,648	-9,223,372,036,854,775,808

■ Gözlemler

- $|TMin| = TMax + 1$
 - Asimetrik sınır değerler
- $UMax = 2 * TMax + 1$

■ C Programlamada

- Limit değerler `#include <limits.h>` içinde tanımlanır
- Aşağıdaki gibi sabitler tanımlanır,
 - `ULONG_MAX`
 - `LONG_MAX`
 - `LONG_MIN`
- Değerler platforma bağlı olarak değişkenlik gösterir.

Unsigned & Signed Nümerik Değerler

X	B2U(X)	B2T(X)
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1

■ Negatif olmayan değerlerde *eşitlik*

- Negatif olmayan değerler aynı bit vektörleriyle ifade edilir

■ *Teklik (Uniqueness)*

- Her bit vektörü tek bir integer değerini ifade eder.
- İfade edilebilen her integer tek bir bit vektörüyle ifade edilir.

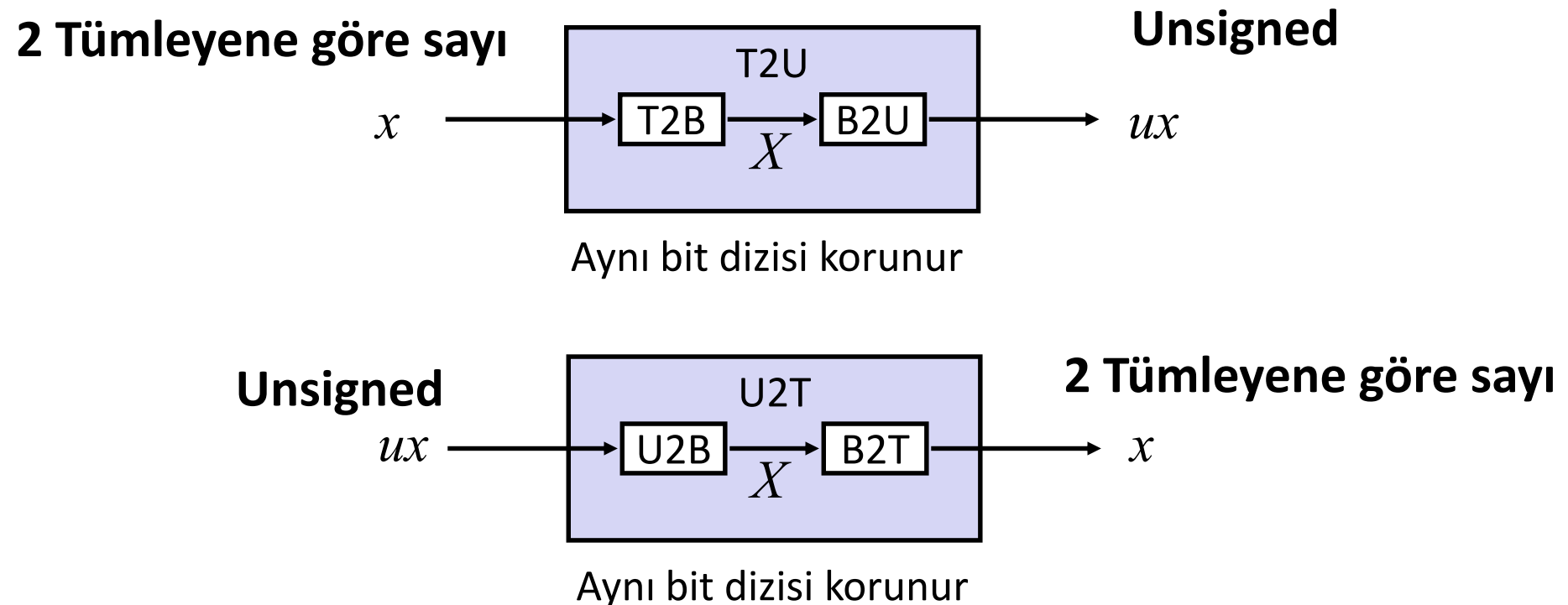
■ Eşleştirmeler tersine çevrilebilir

- $U2B(x) = B2U^{-1}(x)$
 - U2B, x unsigned integer'ının bit vektörünü verir.
- $T2B(x) = B2T^{-1}(x)$
 - T2B, 2 tümleyene göre ifade edilen x integer'ının bit vektörünü verir.

Bit, Byte ve Integer

- Bilginin bitler ile ifade edilmesi
- Bit düzeyinde manipülasyonlar
- **Integer**
 - unsigned ve signed
 - **Dönüştürme (conversion), tip dönüşümü (casting)**
 - Genişletme (expanding), kırpma (truncating)
 - Toplama, işaret değiştirme (negation), çarpma, kaydırma
- Bilginin hafızada saklanma şekilleri, pointer'lar, karakter dizileri (string'ler)

Signed & Unsigned arasında eşleştirmeler



- unsigned ve 2 tümleyene göre verilen işaretli sayılar arasında eşleştirmeler:

Bit dizisini aynen koru ve yeniden yorumla

Signed ↔ Unsigned arası eşleştirme

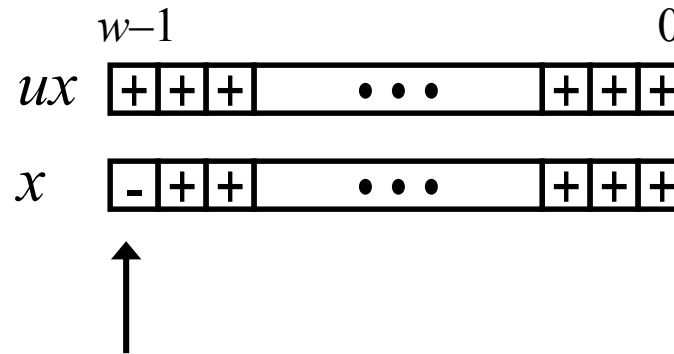
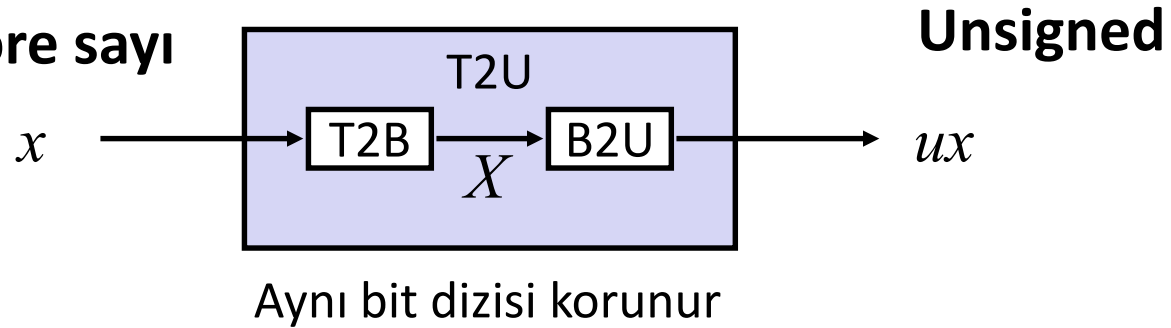
Bit dizisi	Signed		Unsigned
0000	0		0
0001	1		1
0010	2		2
0011	3		3
0100	4		4
0101	5		5
0110	6		6
0111	7		7
1000	-8	→ T2U →	8
1001	-7	← U2T ←	9
1010	-6		10
1011	-5		11
1100	-4		12
1101	-3		13
1110	-2		14
1111	-1		15

Signed ↔ Unsigned arası eşleştirme

Bits	Signed		Unsigned
0000	0	=	0
0001	1		1
0010	2		2
0011	3		3
0100	4		4
0101	5		5
0110	6		6
0111	7		7
1000	-8	+/- 16	8
1001	-7		9
1010	-6		10
1011	-5		11
1100	-4		12
1101	-3		13
1110	-2		14
1111	-1		15

Signed & Unsigned arası ilişki

2 Tümleyene göre sayı

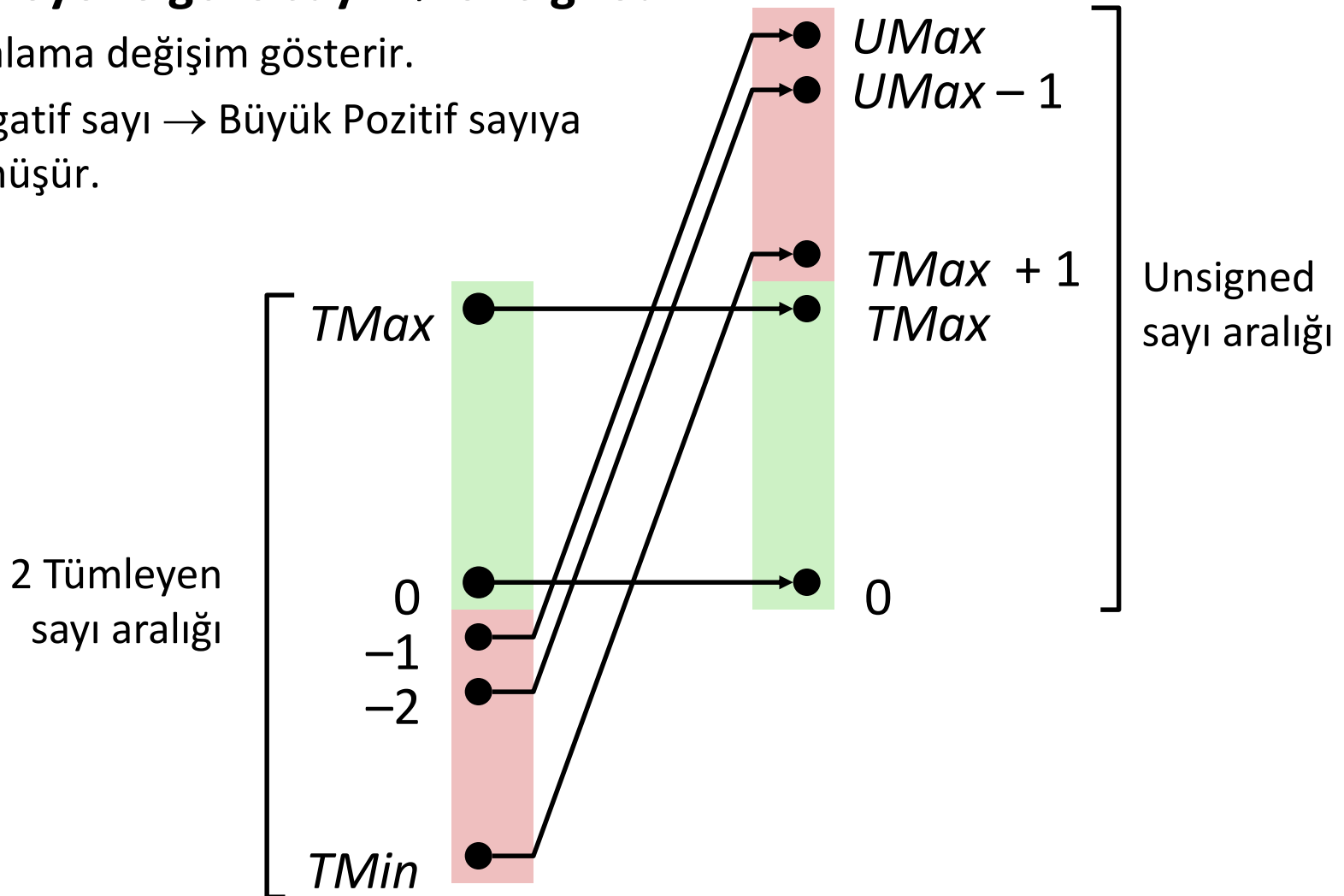


Büyük negatif ağırlık
Büyük pozitif ağırlığa
dönüşür

Dönüştürmenin görsel gösterimi

■ 2 Tümleyene göre sayı → Unsigned

- Sıralama değişim gösterir.
- Negatif sayı → Büyük Pozitif sayıya dönüşür.



C'de Signed ve Unsigned

■ Sabitler

- Varsayılan olarak signed integer kabul edilirler.
- Eğer “U” son eki kullanılmışsa o zaman unsigned’dırlar.

0U, 4294967259U

■ Tip dönüşümü (Casting)

- U2T ve T2U’daki gibi signed & unsigned arasındaki doğrudan, belirgin (Explicit) tip dönüşümü

```
int tx, ty;  
unsigned ux, uy;  
tx = (int) ux;  
uy = (unsigned) ty;
```

- Aynı zamanda atama ve yordam çağrısı (procedure call) ile de dolaylı olarak (Implicit) tip dönüşümü gerçekleşir.

```
tx = ux;  
uy = ty;
```

Tip Dönüşümü (Casting) Süprizleri

■ İfadelerin (Expression) İşletilmesi, Değerlendirilmesi (Evaluation)

- Eğer bir ifade içinde unsigned ve signed karışık olarak bulunuyorsa, ***signed değerler dolaylı olarak unsigned'a dönüştürülür***
- Karşılaştırma işlemlerini de kapsar <, >, ==, <=, >=
- W = 32 için örnekler: **TMIN = -2,147,483,648 , TMAX = 2,147,483,647**

■ Sabit ₁	Sabit ₂	İlişki	Değerlendirme
0	0U	==	unsigned
-1	0	<	signed
-1	0U	>	unsigned
2147483647	-2147483647-1	>	signed
2147483647U	-2147483647-1	<	unsigned
-1	-2	>	signed
(unsigned)-1	-2	>	unsigned
2147483647	2147483648U	<	unsigned
2147483647	(int) 2147483648U	>	signed

Özet

Signed $\leftrightarrow$ Unsigned Tip Dönüşümü: Temel Kurallar

- Bit dizisi korunur
- Fakat yeniden yorumlanır.
- Hiç beklenmedik etkileri olabilir: 2^w kadar büyüklükte ekleme veya çıkarma
- **signed ve unsigned int aynı ifade içinde yer alıyorsa:**
 - `int, unsigned`'a dönüştürülür!!